

FOC 工程执行流程与时序分析

工程: f2800157_bldc_FOC **主控:** TMS320F2800157 @ 120 MHz **电机:** 42JSF630AS-1000 (50极对, 1000线编码器) **驱动:** 野火直流无刷电机驱动板 (三电阻采样, 信号隔离, 12~48V)

一、工程文件结构

user/	
├ main.c	主程序入口 + ISR + 状态机 + 故障检测
├ foc.h	数据结构定义 (FOC_t, PI_t, Encoder_t, 状态枚举)
├ foc.c	核心算法 (Clarke, Park, IPark, SVPWM, PI, 编码器, 位置环)
├ hal.h	HAL 层声明 + inline 读写函数 (电流/电压/PWM/编码器)
├ hal.c	硬件初始化 (GPIO, PWM, ADC, eQEP, 中断)
└ user_config.h	全部可配置参数 (引脚, ADC通道, 电机参数, PI增益, 保护阈值)

二、系统总览：两个执行上下文

整个系统只有**两个执行上下文**，它们并发运行：



关键规则: ISR 可以随时打断主循环, 但主循环不能打断 ISR。

三、上电启动流程 (main 函数)

main()	
├ 1. Device_init()	芯片初始化 (PLL→120MHz, 看门狗, 外设时钟使能)
├ Device_initGPIO()	GPIO 模块时钟使能
├ 2. Interrupt_initModule()	PIE 模块初始化 (清除所有中断)

Interrupt_initVectorTable() 加载默认中断向量表	
3. HAL_init()	硬件外设初始化 (详见下方)
├ HAL_setupGPIO()	PWM引脚(0~5), eQEP引脚(20,21,23), 驱动使能(GPIO28=低=禁用), LED(GPIO31)
├ HAL_setupPWM()	ePWM1/2/3 互补中心对齐, 15kHz, 死区400ns, ePWM1产生SOCA触发ADC
├ HAL_setupADC()	ADCA: Ia(SOC0-3, 4x), Ic(SOC4-7, 4x), Vdc(SOC8) ADCC: Ib(SOC0-3, 4x)
├ HAL_setupEQEP()	中断源: ADCA SOC8完成 → ADCA_INT1 eQEP1: 正交4倍频, 4000counts/rev, Unit Timer 1ms, 捕获模式
├ HAL_setupInterrupt()	注册 motorControlISR → ADCA_INT1
└ HAL_setupCPUTimer()	CPU Timer0: 1ms 周期 (后台定时)
4. FOC_init(&foc)	FOC 结构体全部清零, PI控制器参数初始化
5. EINT / ERTM	使能全局中断 → ISR 开始以 15kHz 运行 (此时 motorState=IDLE, ISR 输出50% PWM)
6. for(;;) 主循环	每 1ms 执行一次:
├ runStateMachine()	状态机管理 (见第五节)
└ LED 闪烁逻辑	运行=200ms闪, 故障=1s闪, 空闲=常亮

四、ISR 执行流程 (motorControlISR, 15kHz)

每 66.7μs 执行一次, 由 ePWM1 SOCA → ADC 转换完成触发。

4.1 ISR 入口: 采样与预处理 (每次都执行)

motorControlISR()	
1. ADC 读取 (三电阻全采样)	
ia_raw = HAL_readCurrentA()	ADCA SOC0-3 平均, 减硬件偏置2048, ×缩放
ib_raw = HAL_readCurrentB()	ADCC SOC0-3 平均, 减硬件偏置2048, ×缩放
ic_raw = HAL_readCurrentC()	ADCA SOC4-7 平均, 减硬件偏置2048, ×缩放
↓	
foc.iABC_A.a = ia_raw - offsetIa	减去软件校准偏置 (初始=0)
foc.iABC_A.b = ib_raw - offsetIb	
foc.iABC_A.c = ic_raw - offsetIc	
2. 母线电压	
foc.vDC_V = HAL_readVdc()	ADCA SOC8, ×电压缩放
if < 6V → 用24V额定值代替 (防止SVPWM除零)	
3. 编码器角度更新	
rawCount = eQEP位置寄存器	
Encoder_run():	
├ thetaMech_rad = count / 4000 × 2π	机械角度
└ thetaElec_rad = (count-offset) % 80 / 80 × 2π	电角度

```

└─ posMech_rad = turnCount×2π + thetaMech 绝对位置(含圈数)
   thetaFOC_rad = thetaElec_rad
4. switch(motorState) → 进入对应状态分支...

```

4.2 状态分支详解

ISR 的核心是一个 switch-case，根据 `motorState` 执行不同逻辑：

```

switch(motorState)
├─ IDLE / STOP / FAULT → 输出 50% PWM（三相等压，电机无力矩）
├─ OFFSET_CAL → 偏置校准（详见 4.3）
├─ ALIGN → 转子对齐（详见 4.4）
└─ RUN → 闭环运行（详见 4.5）

```

4.3 OFFSET_CAL 状态 (ADC 偏置校准)

目的：电机静止、PWM 50% 时，测量 ADC 的零电流残余偏差。

```

持续 2000 次 ISR ≈ 133ms
├─ 每次 ISR：
   offsetSumIa += ia_raw      独立累加器累加（不影响 offsetIa）
   offsetSumIb += ib_raw
   offsetSumIc += ic_raw
   offsetCounter++
   输出 50% PWM（电机无电流）
└─ 第 2000 次：
   offsetIa = offsetSumIa / 2000  最终偏置值生效
   offsetIb = offsetSumIb / 2000
   offsetIc = offsetSumIc / 2000
   → 切换到 MTR_STATE_ALIGN

```

4.4 ALIGN 状态 (转子对齐)

目的：在电角度=0的位置施加 d 轴电流，将转子拉到已知位置，记录编码器偏移。

```

持续 500ms（7500 次 ISR）
├─ 前 200ms：缓慢升流
   alignCurrent: 0 → 1.5A（斜坡，每ISR增加约 0.0005A）

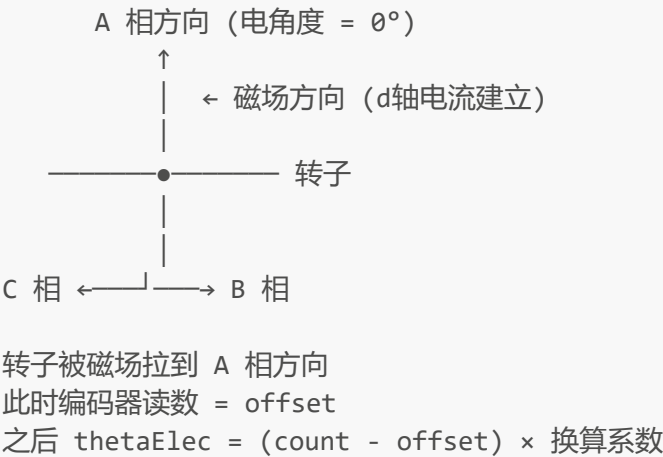
```

— 每次 ISR 执行完整 FOC 链路 (但电角度强制=0):
 sincos(0) → sin=0, cos=1
 Clarke: ia, ib → Iα, Iβ
 Park(θ=0): Iα, Iβ → Id, Iq
 PI(Id): ref=alignCurrent, fbk=Id → Vd
 PI(Iq): ref=0, fbk=Iq → Vq
 IPark(θ=0): Vd, Vq → Vα, Vβ
 SVPWM: Vα, Vβ → duty_a, duty_b, duty_c
 写入 PWM

 效果: 电流闭环控制, 在 A 相方向建立磁场
 转子被拉到 A 相对应位置 (电角度0)

— 第 7500 次 (500ms到达):
 foc.enc.offset = rawCount 记录此位置为电角度零点
 turnCount = 0 圈数清零
 posMech_rad = 0 位置清零
 PI_reset(piId, piIq, piSpd) 清除PI积分器
 → 切换到 MTR_STATE_RUN

对齐原理图:



4.5 RUN 状态 (闭环运行)

— 每次 ISR (15kHz, 电流环):
 sincos(thetaFOC_rad) 用实时电角度
 Clarke: ia, ib → Iα, Iβ
 Park: Iα, Iβ → Id, Iq (用实时角度旋转到 dq 坐标系)

— 每 10 次 ISR (1.5kHz, 速度/位置环):
 编码器速度更新
 HAL_calcEncoderSpeed() M法: eQEP Unit Timer 1ms内的脉冲差
 Encoder_calcSpeed() rpm 和 Hz 单位转换

└ 根据 ctrlMode 计算 iqRef_A:

└ 转矩模式 (mode=0): iqRef = cmdTorque_A (用户直给)

└ 速度模式 (mode=1):

spdRef = cmdSpeed_rpm × 2π/60

iqRef = PI_spd(spdRef, speedMech_rads)

└ 位置模式 (mode=2):

posRef = cmdPosition_rad

spdRef = PosLoop_P(posRef, posMech_rad, spdLimit)

iqRef = PI_spd(spdRef, speedMech_rads)

└ 电流环 PI (每次 ISR):

idRef = 0 (表贴电机, 无弱磁)

Vd = PI_Id(idRef, Id)

Vq = PI_Iq(iqRef, Iq)

└ IPark: Vd, Vq → Va, Vβ (用实时角度旋转回 αβ)

└ SVPWM: Va, Vβ → duty_a/b/c (零序注入法, 电压利用率提升15%)

└ 写入 PWM

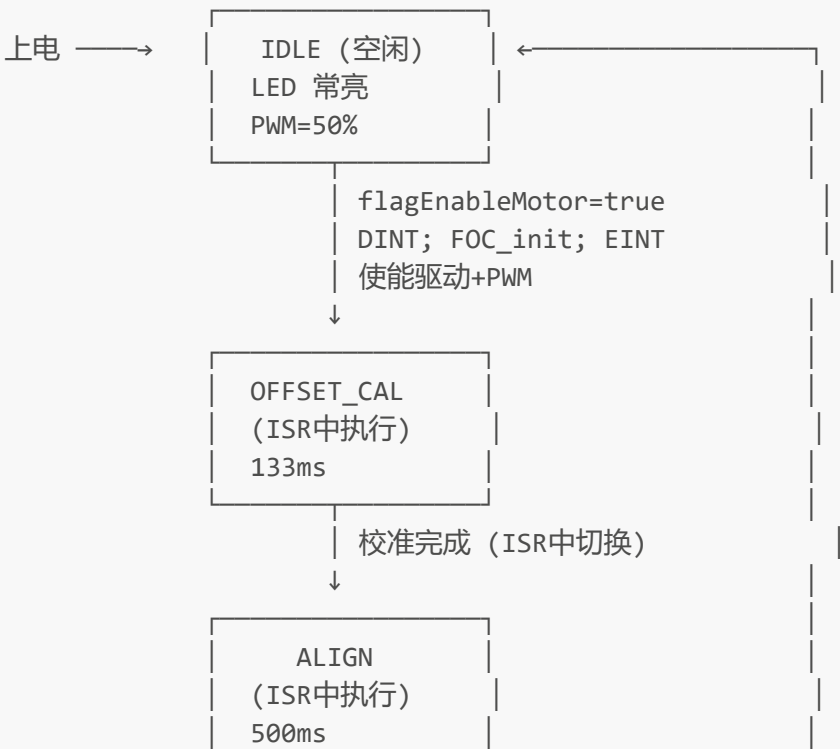
└ 故障检测:

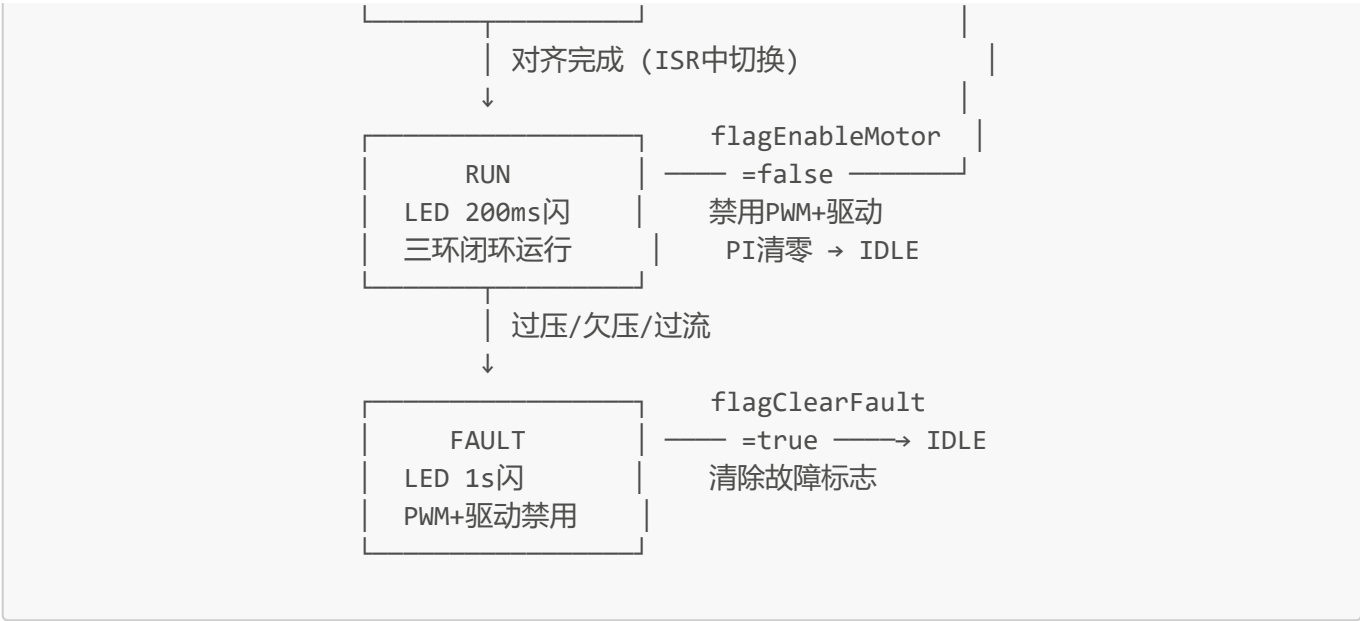
过压 > 50V? 欠压 < 10V? 过流 > 10A?

→ 若有故障: motorState = FAULT, 禁用 PWM

五、后台状态机 (runStateMachine, 1kHz)

在主循环中每 1ms 调用一次，负责**状态转换管理**（不负责控制算法）。



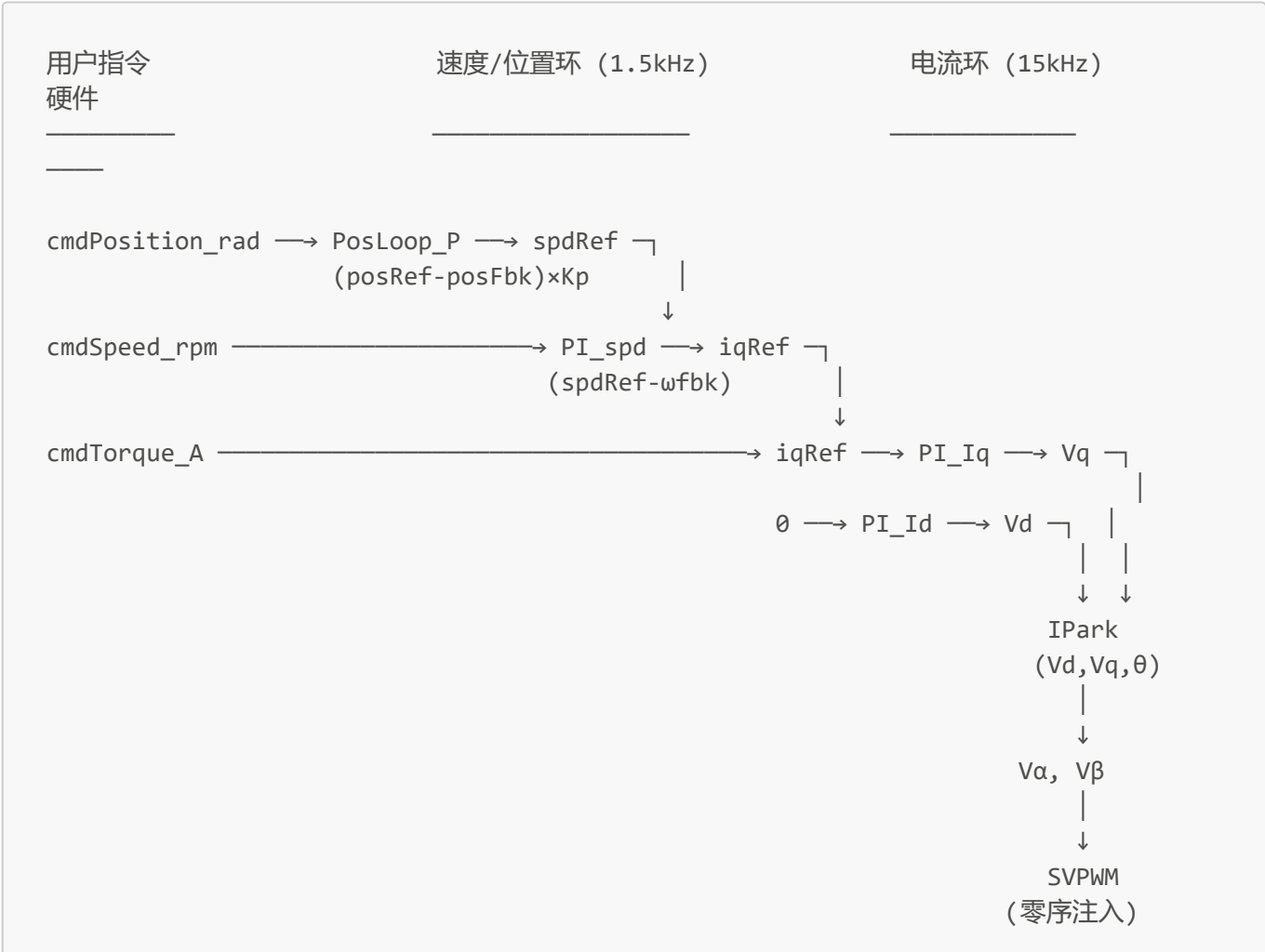


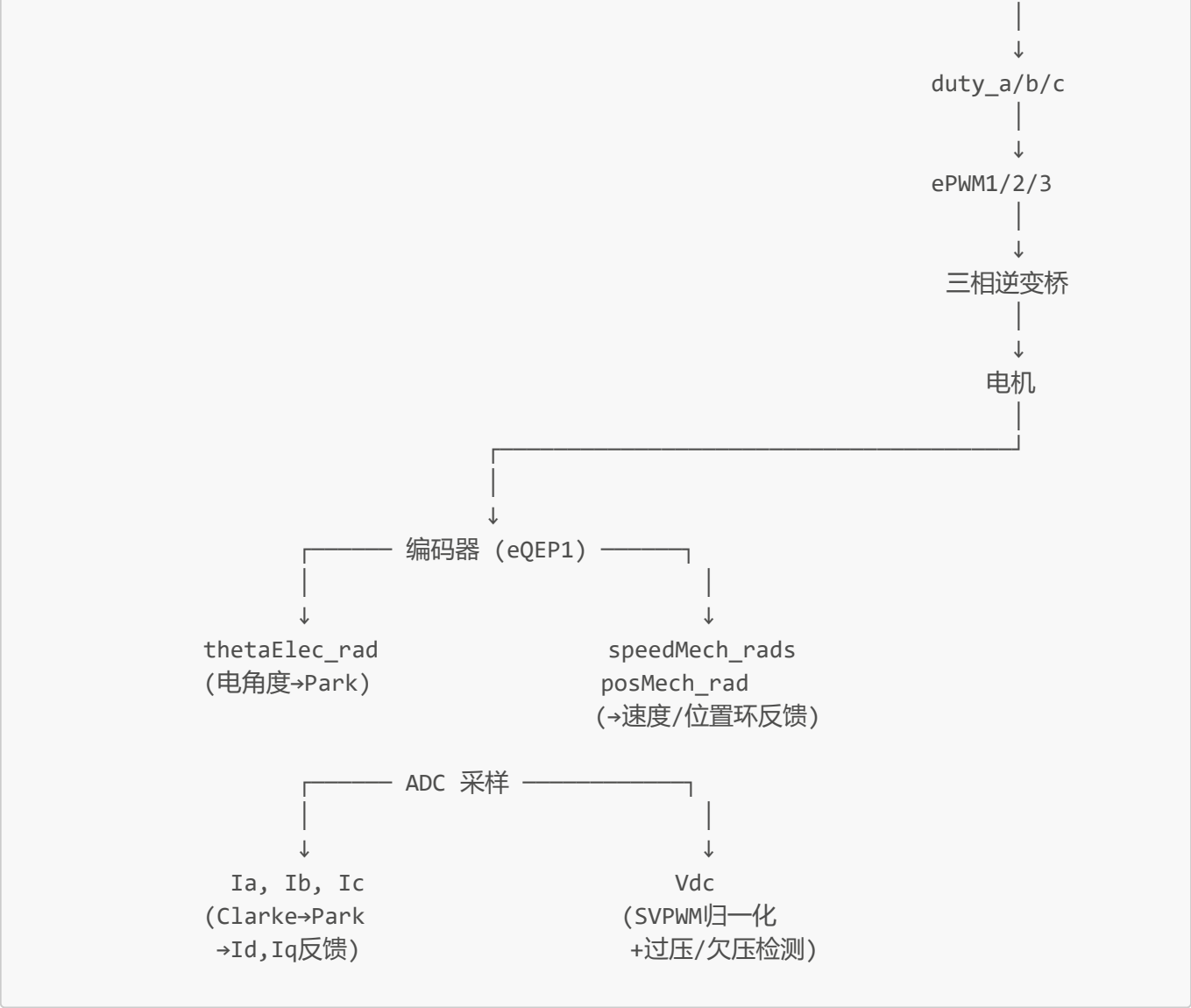
六、完整启动时序图 (时间轴)

时间	ISR (15kHz)	后台 (1kHz)	硬件状态
		main() 开始 Device_init (PLL, WDG) HAL_init (PWM,ADC,eQEP) FOC_init EINT → ISR开始运行	
T=0ms	ISR: IDLE → 50% PWM	主循环等待1ms标志	驱动禁用 电机无力矩
T=1ms	ISR: IDLE → 50% PWM	runStateMachine:IDLE 等待 flagEnableMotor	
:	:	:	
:	(用户在调试器中设 flagEnableMotor=true)		
:	:	:	
T=Xms	ISR: IDLE → 50% PWM	检测到 flagEnableMotor! DINT (禁中断) FOC_init, 偏置清零 EINT (恢复中断) HAL_enableDRV → GPIO28=1 HAL_enablePWMoutput motorState=OFFSET_CAL	
— 偏置校准阶段 (133ms) —			
T+0ms	ISR: OFFSET_CAL 累加 ia/ib/ic_raw 输出 50% PWM	(后台无动作)	驱动使能 PWM=50% 电机无电流
T+1ms	累加继续...		
:	:		
T+133ms	第2000次ISR: offsetIa/Ib/Ic 计算完成 → motorState=ALIGN		

— 转子对齐阶段 (500ms) —			
T+133ms	ISR: ALIGN $\theta=0$, $I_{d_ref}=0.0A$ FOC链路闭环运行	(后台无动作)	电机开始有电流 转子缓慢转动
T+200ms	alignCurrent 升到 0.5A	:	转子的A相方向移动
T+333ms	alignCurrent = 1.5A (满额)	:	转子锁定在A相方向
:	保持1.5A d轴电流	:	转子稳定不动
T+633ms	第7500次ISR: enc.offset = rawCount PI 清零 → motorState=RUN		记录零点
— 闭环运行阶段 —			
T+633ms	ISR: RUN 电流环 15kHz 速度环 1.5kHz (速度模式, 等待速度指令)	(后台监控) LED 200ms闪	电机受控运行
(用户设 cmdSpeed_rpm=100) 速度环PI → iqRef → 电流环 → PWM → 电机加速到100rpm			

七、控制环数据流 (RUN 状态)





八、关键时间参数汇总

参数	值	说明
ISR 频率	15 kHz (66.7μs)	电流环执行频率
电流环带宽	~1 kHz	PI: Kp=3.71, Ki=0.427
速度环频率	1.5 kHz (667μs)	10:1 分频
速度环带宽	~100 Hz	PI: Kp=0.10, Ki=0.005
位置环频率	1.5 kHz	与速度环同步
位置环带宽	~30 Hz	P: Kp=50.0
偏置校准	133ms	2000 次平均
对齐时间	500ms	前200ms升流, 后300ms保持
升流斜率	1.5A / 200ms	平滑, 避免机械冲击
死区	400ns	ePWM 互补输出

参数	值	说明
ADC 过采样	4x	噪声降低 2 倍 (6dB)
编码器分辨率	4000 counts/rev	1000线 × 4倍频
电角度分辨率	4.5°	80 counts/电周期
速度测量周期	1ms	eQEP Unit Timer
故障检测	15 kHz	每次 ISR 都检查
后台循环	1 kHz	CPU Timer 1ms

九、调试操作指南 (CCS Expressions 窗口)

启动电机

```
flagEnableMotor = true
cmdCtrlMode = 1
cmdSpeed_rpm = 100.0
```

→ 触发: IDLE → OFFSET_CAL → ALIGN → RUN
→ 速度模式
→ 目标速度 100rpm

切换控制模式

```
cmdCtrlMode = 0
cmdCtrlMode = 1
cmdCtrlMode = 2
```

→ 转矩模式, 使用 cmdTorque_A
→ 速度模式, 使用 cmdSpeed_rpm
→ 位置模式, 使用 cmdPosition_rad

停止电机

```
flagEnableMotor = false
```

→ RUN → IDLE (禁用PWM和驱动)

清除故障

```
flagClearFault = true
```

→ FAULT → IDLE (需重新启动)

观测变量

```
foc.iDQ.d / foc.iDQ.q
foc.enc.speedMech_rpm
foc.enc.posMech_rad
foc.vDC_V
```

d/q 轴实际电流 (A)
实际转速 (rpm)
绝对位置 (rad)
母线电压 (V)

faultFlags.all	故障标志 (0=正常)
motorState	当前状态 (0~5)